Rubric and readiness assessment

Group 15 • Final assessment preparation • 3 November 2026

The rubric rewards a working system under load and an explanation of why it behaves that way. Group 15 has strong evidence for data correctness, durable processing, and a successful local workload. Excellent is not yet established for the professor's variable workload, and restoring the stopped deployment is the first practical priority.

The official assessment

The Group Assignment Rubric INFOMCEC 2026–2027, page 1, distinguishes the following bands. These are paraphrases of the rubric, not additional grading rules.

Component	Fair	Good	Excellent
Final demo, 70%	Working implementation and consistent data meeting the interfaces and requirements	Also passes variable workload stress with notification latency below 10 seconds	Also keeps up with the variable HTTP workload
Architecture assessment, 30%	All interfaces implemented, data consistency, and separation of concerns	Distinct responsibilities and separation based on scalability	Also pinpoints different bottlenecks when load is applied

A working demo is a prerequisite for the assignment to be graded. Individual grades can differ because of contribution and knowledge. Questions may be redirected to any group member; specialization does not excuse an inability to explain another service.

The rubric's resit note says an additional resit beyond the midterm/final opportunities, if granted, is restricted to a passing grade for both demo and architecture. If that special situation applies to your group, preparation cannot remove the policy ceiling. The ordinary final assessment is the target assumed here.

The rubric does not give a numeric score conversion for each band. Do not calculate a predicted final grade from these descriptors.

Criterion by criterion

Excellent-level requirement	Available evidence	Current position and next action
Demo works	October local readiness, instructor validation, and connection artifacts	**Blocked operationally now:** zero containers after cleanup. Restore and verify before relying on the demo
Consistent data	202 complete endpoint comparisons in the final instructor run; transaction/replay/cache tests	Strong historical evidence. Repeat integration and instructor correctness checks after recovery
Below 10 seconds under variable load	All 610 notifications in the final local instructor run, maximum 5.641 seconds	Proven for one stated load profile. Add changing rates/bursts and strict maximum checks
Keeps up with variable HTTP load	97.310 achieved against 100 offered HTTP/s in that run	Strong baseline, incomplete variable-rate evidence; earlier 200-HTTP/s profile missed the local throughput criterion
All external interfaces	Kafka Avro input, required history API, notification API	Implemented. Recheck current course connectivity and external API reachability after recovery
Distinct responsibilities	Scheduler, stateless executors, Data, gateway, notifier	Explain ownership and independent scaling, plus the extra serialization/network cost
Pinpoints different bottlenecks	State-write round trips led to a specific fix; replica matrix exposed a shared-resource ceiling	Present these observations and show how metrics distinguish CPU, database, partition, cache, and notification constraints
Individual understanding	Source, test artifacts, worked cases, question bank	Each member must demonstrate it in rehearsal; reports cannot establish individual competence

Sources: final stress, failed stress, sizing matrix, current audit.

What the current results support

A defensible statement is:

> *We implemented all required interfaces and preserved consistent histories in our tests. On a two-vCPU VM, our final local run of the instructor's original tools handled 100 experiments with 10 sensors at 1 Hz, delivered all 610 required notifications within 5.641 seconds, and achieved 97.310 successful HTTP requests per second against 100 offered. We can explain both the database optimization that improved this run and the shared-resource limit observed in other profiles.*

The final run used one completed warmup before the 100 concurrent experiments. Its 202 history comparisons equal two endpoints times 101 total completed experiments. The average number of notifications varies with the simulator's random temperature sequence.

It would be inaccurate to say that all professor workloads have passed, that 200 HTTP/s is sustainable on this host, that the service is currently live, or that external notification delivery is exactly once.

Priority actions

Priority 0 Restore the demonstration

Recover the core deployment with the same Compose project and named volumes. Verify PostgreSQL readiness and old experiment data before making stronger preservation claims. Keep enough disk headroom for rebuilding and retained data; the cleanup left approximately 3.5–3.9 GiB available depending on observation time.

Check API health, Kafka partition assignment, outbox status, and the selected topic/destination. For the professor's demo these are the shared experiment topic and the external notifications endpoint. A local

grading-panel pass uses a different topic and destination.

Done means: a fresh readiness record, successful required endpoint queries, correct configuration, and current course preflight. Follow the recovery runbook. Container creation alone does not complete this action.

Priority 1 Close the workload evidence gap

Repeat the original passing profile after recovery, then vary event and HTTP rates over time and retain raw results. Test a rising step, a burst, a sustained plateau, and a return to baseline. Examine whether queue depth returns to its former level rather than growing across repeated steps.

Retest the older failing 200-HTTP/s and twenty-sensor combined profiles on the final implementation. Those October 7 runs predate the October 9 batch-write optimization. They establish earlier limits, not a measured post-optimization ceiling.

Use strict notification maximum **less than 10 seconds** and zero missing/invalid notifications. The existing custom harness checks maximum less than or equal to 10, and the course checker counts delays greater than 10. Exactly 10 is therefore accepted by those local gates but not by the rubric's wording. The successful recorded runs are below 10, so this does not invalidate their actual values. Tighten a future automated gate or explicitly check maximum less than 10 when evaluating a new run.

Our 95% achieved-HTTP threshold is an internal acceptance rule; the professor has not specified that exact percentage. Show offered and completed rates, validation errors, latency, queue trend, and recovery time rather than treating the local boolean as the whole grading decision.

Done means: changing-load traces with correct data, strict latency results, and evidence that requests and queues keep up on the selected deployment.

Priority 2 Make bottleneck explanations observable

For each tested load step, align event rate, HTTP rate, queue length, CPU, executor occupancy, Data latency, outbox age, and validation outcome on one time window. State a hypothesis, identify a metric that would support it, make one targeted change, and compare.

The existing metrics show useful symptoms. Detailed SQL statement timing, pool-wait timing, complete Kafka lag, and per-partition hot-spot monitoring are not all implemented as dedicated metrics. Use database/consumer inspection when needed, or add those instruments as future work. Do not describe planned instruments as existing dashboards.

Done means: all three students can distinguish at least two bottlenecks and explain why the same scaling action does not fix both.

Priority 3 Prevent another storage outage

The disk incident stopped every service, even though named volumes survived. Separate regenerable images/build cache from valuable database and monitoring volumes. Preserve data before cleanup, define storage headroom, measure growth, and alert on filesystem usage. A bounded Redis cache does not bound PostgreSQL history, outbox, Kafka, Docker cache, or monitoring storage.

The current application has no history archival policy or filesystem-full alert rule. Propose a policy compatible with the requirement to retain an experiment's history; deleting older measurements merely to recover space would change product behavior.

Done means: an agreed preservation/recovery process, a monitored capacity threshold, and sufficient space for the demo and its test evidence.

Priority 4 Prove individual understanding

Use the random redirection drills in the team plan. Each member should draw the architecture, calculate a mean/notification example, explain a crash window, read one performance result, identify a bottleneck, and state a limitation.

Record actual contributions with file/commit/test evidence. Assigning someone a presentation topic now does not prove they implemented it earlier. Discuss genuine study, validation, analysis, documentation, operations, and code contributions accurately.

Improvements ordered by value and risk

Improvement	Why it helps the rubric	Evidence required before claiming success
Restore current deployment	Mandatory working-demo gate	Fresh health, connectivity, correctness, and live-mode checks
Strict deadline gate	Matches the actual wording	Boundary case at exactly 10 rejected; measured maximum still below 10
Repeat and vary offered rates	Directly targets Excellent demo	Multiple traces including rise, plateau, burst, and drain
Diagnose database versus CPU pressure	Directly targets Excellent architecture	Correlated metrics and a controlled intervention
Disk monitoring and preservation plan	Prevents another demo outage	Capacity trend, alert check, recovery rehearsal
Calibrate cost inputs	Strengthens the resource/cost argument	Actual billed rates and an explicit scope for the estimate
More executors	Helps only when executor work is limiting	Improved combined latency/throughput without downstream overload
Multiple Data processes	Potential future capacity, with correctness risk	Shared cache invalidation or removal of local metadata cache, plus concurrency tests
Larger or separated infrastructure	Potential resource relief, with cost tradeoff	Confirm allowed deployment constraints; compare performance and cost

The goal is a convincing chain from requirement to design to measurement to revision. Additional components or pages do not compensate for a stopped demo or unsupported performance claims.